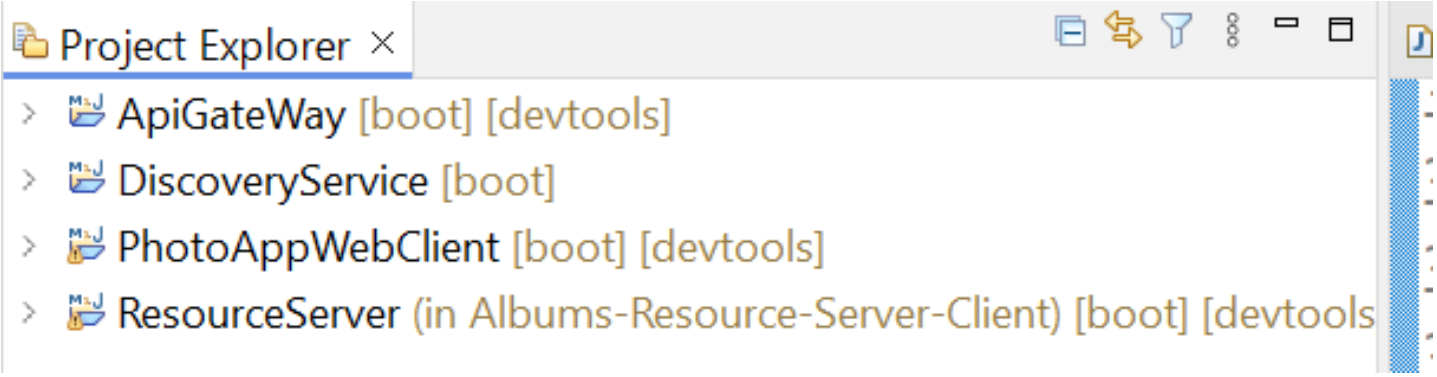
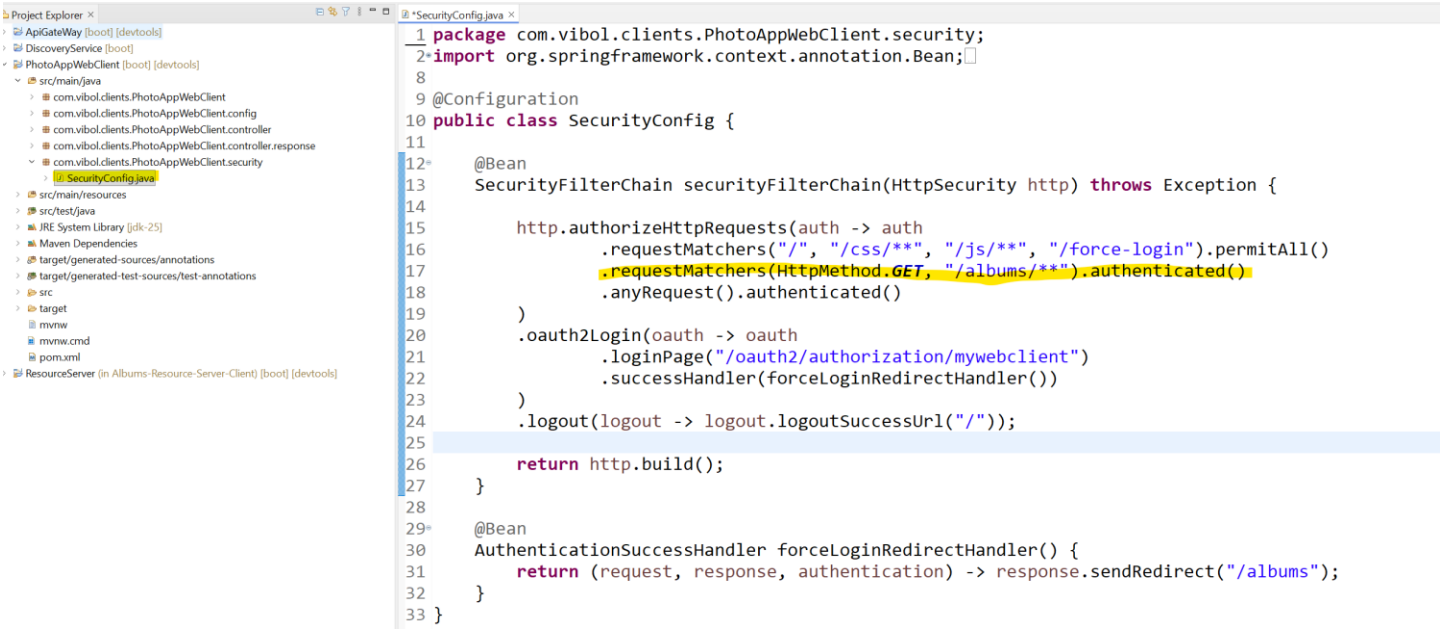
OAuth2 Login + API Call-> Template

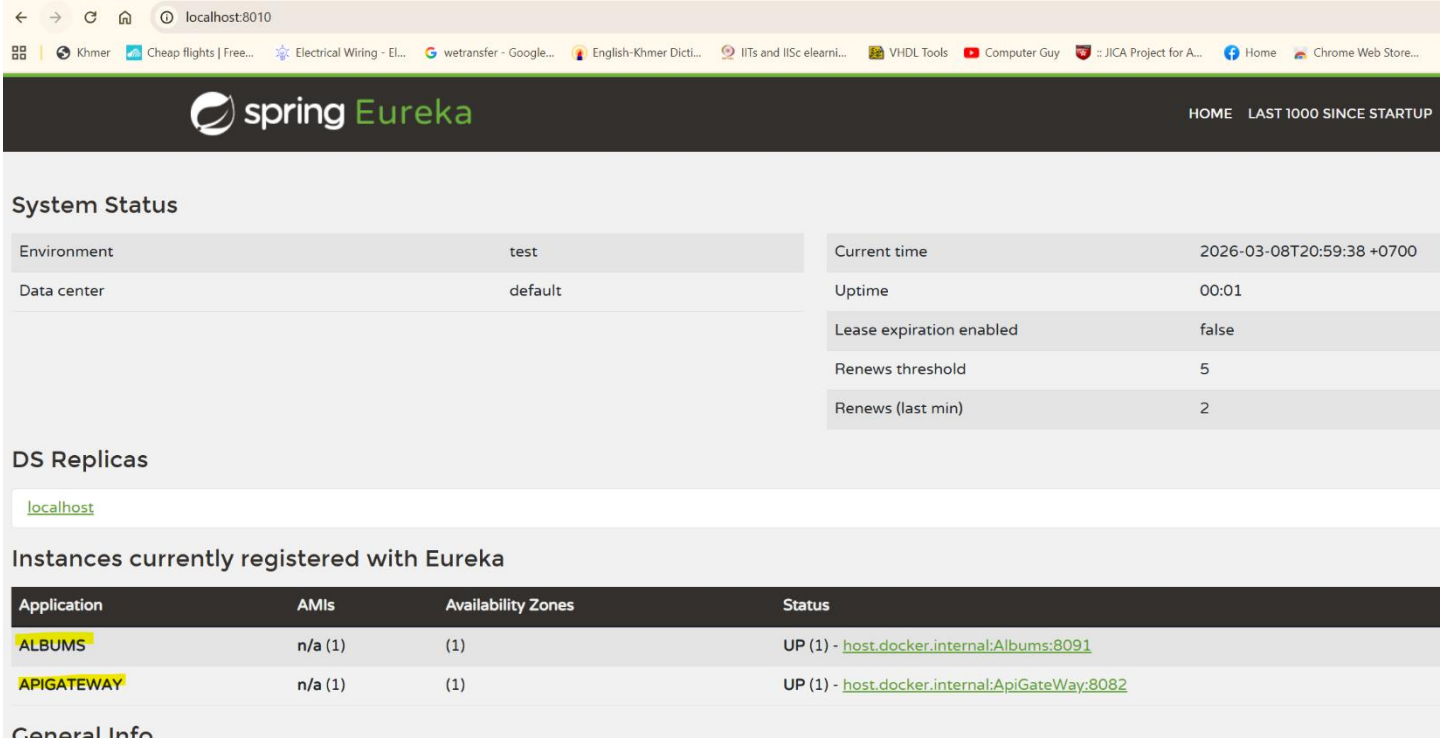
យើងប្រើប្រាស់ ApiGateWay ដើម្បីធ្វើការ Route ទៅកាន់ Resource Server ដោយធ្វើការ Login to the Keyload server to get JWT then can access to Resource Server។



1- In the PhotoAppWebClient (Resource Server)



អោយតាមាន HttpRequest ណាចូលមកត្រូវធ្វើការ ផ្ទៀងផ្ទាត់សិន ត្រូវធ្វើការ Login with Keyload ដើម្បីធ្វើការ AccessToken



Eureka : have the service Albums and APIGateWay

Full Flow (Easy Diagram)

```
1 Browser
    |
    ▼
2 Spring Web Client
    |
    ▼
3 Redirect to Keycloak
    |
    ▼
4 User Login
    |
    ▼
5 Keycloak returns Tokens
    |
    ▼
6 Spring stores OAuth2AuthorizedClient
    |
    ▼
7 Controller gets Access Token
    |
    ▼
8 Call API Gateway with Bearer Token
    |
    ▼
9 API Gateway → Albums Service
    |
    ▼
10 HTML returned to browser
```

1️⃣ Realm Roles (Global Roles)

Realm roles are roles for the entire system (the whole realm).

Think of them as **company-wide roles**.

Example:

```
Realm: mywebclient
```

Users in the system might have roles like:

```
admin
developer
manager
user
```

These roles can be used by **any application** inside the realm.

Example

User:

```
vibollong
```

Realm role:

```
developer
```

Token will contain:

```JSON
"realm_access": {
  "roles": [
    "developer"
  ]
}
```

2️⃣ Client Roles (Application Roles)

Client roles belong to a specific application (client).

Think of them as **roles only inside one app**.

Example apps:

```
photo-app-webclient
order-service
payment-service
admin-dashboard
```

Each app can define its own roles.

Example:

```
Client: photo-app-webclient
Roles:
  developer
  viewer
  uploader
```

These roles are **only for that client**.

Token will contain:

```JSON
"resource_access": {
  "photo-app-webclient": {
    "roles": [
      "developer"
    ]
  }
}
```

3️⃣ Simple Analogy

Imagine a **company building**.

Realm roles = company roles

```
CEO
Manager
Developer
HR
```

These roles work **everywhere in the company**.

Client roles = department roles

Department: **IT system**

```
code-writer
tester
reviewer
```

Department: **Finance system**

```
accountant
auditor
```

These roles only work **inside that department**.

4️⃣ Visual Comparison

Feature	Realm Role	Client Role
Scope	Whole system	One application
Stored in JWT	`realm_access.roles`	`resource_access.client.roles`
Easier for microservices	✅ Yes	⚠️ Harder
Good for	system permissions	app-specific permissions

8️⃣ Quick Rule

Use **Realm Roles** when:

```
roles apply to many services
```

Use **Client Roles** when:

```
roles are only for one application
```

Simple Memory Version (4 Steps)

1 User logs in with Keycloak

Spring Security redirects the user to Keycloak.

```
Browser → Spring App → Keycloak Login
```

Config that enables it:

```java
.oauth2Login(oauth -> oauth
    .loginPage("/oauth2/authorization/mywebclient")
)
```

After login, Keycloak sends tokens back to Spring:

```
Access Token
ID Token
```

---

2 Spring stores the tokens for the user

Spring Security automatically saves the token in an **OAuth2AuthorizedClient**.

That is why we use:

```java
@RegisteredOAuth2AuthorizedClient("mywebclient")
OAuth2AuthorizedClient authorizedClient
```

Spring then gives us the token directly.

---

3 Get the Access Token

Inside the controller:

```java
String accessToken = authorizedClient.getAccessToken().getTokenValue();
```

Important rule to remember:

⚠️ **Access Token → used to call APIs**

⚠️ **ID Token → used for login identity only**

4️⃣ Call API Gateway with the Access Token

Your Spring MVC app calls the API gateway like this:

```
Spring Web Client → API Gateway → Albums Service
```

Send the token in the header:

```java
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(accessToken);
```

Result:

```
Authorization: Bearer <access_token>
```

Now the API Gateway trusts the request because it sees the valid token.

3 Key Things to Always Remember

1️⃣ Use `@RegisteredOAuth2AuthorizedClient`

Instead of manually loading the client.

```java
@RegisteredOAuth2AuthorizedClient("mywebclient")
OAuth2AuthorizedClient authorizedClient
```

2️⃣ Use Access Token for APIs

Correct:

```
Authorization: Bearer ACCESS_TOKEN
```

Wrong:

```
Authorization: Bearer ID_TOKEN ❌
```

3️⃣ Registration ID vs Client ID

Example:

```
registration id = mywebclient
client id       = photo-app-webclient
```

Use **registration id** here:

```java
@RegisteredOAuth2AuthorizedClient("mywebclient")
```

One Sentence to Remember Everything

👉 **Login with Keycloak → Spring stores tokens → Controller gets Access Token → send it as Bearer token to API Gateway.**